NexusPOS – Retail POS System

Low-Level Design (LLD) Document

Project	NexusPOS – Retail Point of Sale & Store Management System
Document Type	Low-Level Design (LLD)
Version	1.0
Target Framework	.NET 10.0 (ASP.NET Core)
Frontend	Angular 21
Status	Released for Review
Date	15 April 2026

Table of Contents

1. Project Structure and Namespace Conventions

1.1 Solution Layout

The solution root contains the following projects, each compiled independently as a standalone ASP.NET Core Web API:

Project / Folder	Type	Description
IdentityService	ASP.NET Core Web API	Authentication, authorization, RBAC, JWT, OTP, Google OAuth
CatalogService	ASP.NET Core Web API	Product and category management, stock tracking
OrdersService	ASP.NET Core Web API	Billing, cart management, checkout saga
AdminService	ASP.NET Core Web API	Store management, staff, inventory, dashboard, reports
PaymentService	ASP.NET Core Web API	Payments, Razorpay integration, saga command handling
ReturnsService	ASP.NET Core Web API	Return workflows and return saga
NotificationService	ASP.NET Core Web API	SignalR hub, notification persistence
ApiGateway	ASP.NET Core Web API	Ocelot reverse proxy / API gateway
RetailPOS.Common	Class Library	Shared authorization, logging
RetailPOS.Contracts	Class Library	Shared MassTransit message contracts
pos-ui	Angular 21 SPA	Frontend POS application

1.2 Internal Structure per Service

Each microservice follows an identical internal structure:

Folder	Contents
Controllers/	API controllers inheriting ControllerBase; routes defined with [Route] and [Http*] attributes
Services/	Business logic classes implementing IService interfaces; injected via DI
Repositories/	Data access classes implementing IGenericRepository<T> and domain-specific interfaces

Folder	Contents
Interfaces/	Contracts for services, repositories, and providers
Models/	EF Core entity classes with data annotations
DTOs/	Data Transfer Objects for request/response serialization
Data/	DbContext classes, DbContextFactory (for migrations), seed data, initializers
Consumers/	MassTransit IConsumer<T> implementations for RabbitMQ message handling
Sagas/	(Where applicable) MassTransitStateMachine<T> and SagaStateMachineInstance<T> classes
Middleware/	Custom ASP.NET Core middleware (ExceptionMiddleware)
Exceptions/	Domain-specific exception types
Properties/	launchSettings.json for dev configuration

2. Class-Level Design

2.1 IdentityService

2.1.1 Models

Class	Key Properties	Relationships
User	Id (PK), Email, PasswordHash, EmployeeCode, Otp, OtpExpiry, IsEmailVerified, VerificationOtp, VerificationOtpExpiry, RefreshToken, RefreshTokenExpiry	Has many UserStoreRole
Role	Id (PK), Name	Has many RolePermission, UserStoreRole
Permission	Id (PK), Code (e.g. 'orders:create'), Description	Has many RolePermission
RolePermission	Id (PK), RoleId (FK), PermissionId (FK)	Bridge table: Role <-> Permission
UserStoreRole	Id (PK), UserId (FK), StoreId (nullable FK), RoleId (FK)	Bridge table: User <-> Store <-> Role; null StoreId = Global Admin
Store (Identity)	Id (PK), StoreCode, Name	Used for store-scoped JWT claims

2.1.2 Key Interfaces

Interface	Methods
IAuthService	LoginAsync(LoginDto), VerifyLoginOtpAsync(VerifyLoginOtpDto), RegisterAsync(RegisterDto), VerifyEmailAsync(email, otp), ResendVerificationOtpAsync(email), SendOtpAsync(email), ResendLoginOtpAsync(email), ResetPasswordAsync(ResetPasswordDto), GoogleLoginAsync(idToken, storeId, role), RefreshTokenAsync(TokenRequestDto), LogoutAsync(email), GetStoresAsync(), GetUsersAsync(), SyncAllUsersAsync()
IEmailService	SendEmailAsync(to, subject, body)
IUserRepository	GetWithRolesByEmailAsync(email), GetWithRolesByIdAsync(id) + IGenericRepository<User>
IStoreRepository	IGenericRepository<Store>

2.1.3 AuthService – Method Details

Method	Input	Output	Description
LoginAsync	LoginDto {Email, Password}	AuthResult {Success, Message='OTP_SENT'}	Validates credentials, auto-migrates legacy plain-text passwords to BCrypt, generates 6-digit OTP (10-min expiry), sends via email. Returns OTP_SENT signal.
VerifyLoginOtpAsync	VerifyLoginOtpDto {Email, Otp}	AuthResult with JWT + RefreshToken	Validates OTP, selects primary UserStoreRole (Admin=null store first), generates JWT with permission claims, generates refresh token (stored in DB + cookie).
RegisterAsync	RegisterDto {Email, Password}	bool	Creates user with BCrypt hash + EmployeeCode, sends verification OTP, publishes UserRegisteredEvent.
VerifyEmailAsync	email, otp	bool	Validates VerificationOtp, sets IsEmailVerified=true.
GoogleLoginAsync	idToken, storeId, role	AuthResult	Validates Google ID Token, auto-creates user if new, resolves UserStoreRole, issues JWT + refresh token.
RefreshTokenAsync	TokenRequestDto {AccessToken, RefreshToken}	AuthResult	Validates expired JWT principal, looks up refresh token in DB, rotates refresh token, issues new JWT.
GenerateJwt (private)	User, roleName, permissions, storeId, storeCode, shiftDate	JWT string	Builds claims list (sub, email, role, StoreId, StoreCode, ShiftDate, permission[], aud[]), signs with HMAC-SHA256, 1-hour expiry.
SyncAllUsersAsync	(none)	int count	Publishes UserRegisteredEvent for all users (administrative sync tool).

2.2 CatalogService

2.2.1 Models

Class	Key Properties	Notes
Product	Id, Sku, Barcode, Name, MRP (decimal 18,2), SellingPrice (decimal 18,2), TaxCode, ReorderLevel, IsActive, StockQuantity, CategoryId (FK), StoreId	StoreId used for tenant isolation via global query filter
Category	Id (manual PK, not auto-generated), Name, Description, IsActive, StoreId	Products collection navigation

2.2.2 Key Interfaces and Services

Interface	Implementation	Key Methods
IProductService	ProductService	GetAllAsync(), GetByIdAsync(id), CreateAsync(dto), UpdateAsync(id, dto), DeleteAsync(id), SearchAsync(term)
ICategoryService	CategoryService	GetAllAsync(), GetByIdAsync(id), CreateAsync(dto), UpdateAsync(id, dto), DeleteAsync(id)
IProductRepository	ProductRepository	GetWithCategoryAsync(id), FindByStoreAsync(storeId) + IGenericRepository<Product>
ICategoryRepository	CategoryRepository	IGenericRepository<Category>
ITenantProvider	TenantProvider	GetStoreId() – extracts StoreId from JWT claim via IHttpContextAccessor

2.2.3 MassTransit Consumers (CatalogService)

Consumer	Message Type	Action
CategoryConsumer	CategoryCreatedEvent	Creates a Category record in CatalogDb for the new store category published by AdminService
SagaDeductStockConsumer	DeductStockCommand	Decrements StockQuantity for each item in the command. Publishes StockDeductedEvent on success or StockDeductionFailedEvent on error.
StockAdjustedConsumer	StockAdjustedEvent	Applies manual inventory adjustment to product StockQuantity in CatalogDb.
OrderReturnedConsumer	OrderReturnedEvent	Restocks product quantities when a return is finalized. Publishes StockRestockedEvent.

2.3 OrdersService

2.3.1 Models

Class	Key Properties	Notes
Bill	Id, BillNumber, Date, StoreId, CashierId, CustomerMobile, TotalAmount (decimal 18,2), TaxAmount (decimal 18,2), Status ('Draft'|'Held'|'Finalized'), CustomerName	Status lifecycle: Draft -> Held/Finalized
BillItem	Id, BillId (FK), ProductId, ProductName, UnitPrice (decimal 18,2), Quantity, SubTotal (decimal 18,2), StoreId	Child collection of Bill
Customer	Id, Mobile (10-digit regex validated), Name, CreatedAt, StoreId	Linked by mobile number

2.3.2 Key Services

Interface	Implementation	Key Methods
IBillService	BillService	GetAllBillsAsync(), GetBillByIdAsync(id), CreateOrUpdateCartAsync(dto) – upserts Draft bill with items, FinalizeBillAsync(id) – transitions to Finalized + publishes CheckoutInitiatedEvent, HoldBillAsync(id)
ICustomerService	CustomerService	GetAllAsync(), GetByMobileAsync(mobile), CreateAsync(dto), UpdateAsync(id, dto)

2.3.3 Checkout Saga (CheckoutStateMachine)

The CheckoutStateMachine is a MassTransitStateMachine<CheckoutSagaState> persisted to OrdersSagaDbContext in SQL Server. State transitions:

State	Trigger Event	Action(s) Performed	Next State
(Initial)	CheckoutInitiatedEvent	Populate saga fields (OrderId, StoreId, TotalAmount, Items). Publish ProcessPaymentCommand.	PaymentProcessing
PaymentProcessing	PaymentProcessedEvent (Success)	Publish DeductStockCommand with order items.	StockDeductionProcessing
PaymentProcessing	PaymentProcessedEvent (Failure)	Log failure.	Failed (Finalized)
StockDeductionProcessing	StockDeductedEvent	Publish FinalizeOrderCommand + OrderPlacedEvent.	Completed (Finalized)

State	Trigger Event	Action(s) Performed	Next State
StockDeductionProcessing	StockDeductionFailedEvent	Publish RefundPaymentCommand.	Refunding
Refunding	PaymentRefundedEvent	Log compensation complete.	Failed (Finalized)

2.3.4 CheckoutSagaState Fields

Field	Type	Purpose
CorrelationId	Guid	MassTransit saga correlation key
CurrentState	string	Current saga state name
OrderId	int	Links saga to the Bill record
StoreId	int	Store context
TotalAmount	decimal	Payment amount to process
CustomerMobile	string?	Optional customer mobile
PaymentId	int?	Set after payment for refund reference
Items	List<SagaOrderItem>	ProductId + Quantity for stock deduction
CreatedAt / UpdatedAt	DateTime	Audit timestamps

2.4 AdminService

2.4.1 Models

Class	Key Properties	Notes
AdminStoreEntity	Id, StoreCode, Name, Location, IsActive	Soft-delete via IsActive; global query filter applies
StaffMember	UserId (manual PK = IdentityService UserId), Email, FullName, IsAssigned, AssignedStoreId, AssignedRole, RegisteredDate	Synced from IdentityService via UserRegisteredEvent
InventoryAdjustment	Id, ProductId, Quantity (positive/negative), Reason, StoreId, Date	Records manual stock adjustments
DashboardModels (DTO)	DashboardDto, RecentBillDto, HourlyTrendDto, LowStockItemDto, CategoryBreakdownDto	Query result shapes for dashboard endpoint

2.4.2 Key Services

Interface	Implementation	Key Methods
IDashboardService	DashboardService	GetDashboardAsync(storeId?) – aggregates SyncedOrders, SyncedReturns, InventoryAdjustments from AdminDb. Calculates today/yesterday sales, change %, low stock count, hourly trend, store breakdown.
IInventoryService	InventoryService	GetInventoryAsync(), AdjustStockAsync(dto) – records InventoryAdjustment and publishes StockAdjustedEvent
IReportService	ReportService	GetSalesReportAsync(from, to), GetTaxReportAsync(from, to) – proxies to OrdersService /api/bills with JWT forwarding

2.4.3 Onboarding Saga (OnboardingStateMachine)

State	Trigger Event	Action(s)	Next State
(Initial)	UserRegisteredEvent	Populate saga (UserId, Email). Publish CreateStaffProfileCommand.	ProfileCreationPending
ProfileCreationPending	StaffProfileCreatedEvent	Log completion.	Completed (Finalized)

2.5 PaymentService

2.5.1 Models

Class	Key Properties	Notes
Payment	Id, BillId, PaymentMode ('Cash'|'Card'|'UPI'), Amount (decimal 18,2), ReferenceNumber, StoreId	ReferenceNumber holds Razorpay payment ID for UPI/Card

2.5.2 Key Interfaces and Services

Interface	Implementation	Key Methods
IPaymentService	PaymentService	ProcessPaymentAsync(payment) – saves Payment, publishes PaymentProcessedEvent. GetStorePaymentsAsync(storeId).
IRazorpayService	RazorpayService	CreateOrderAsync(RazorpayOrderRequest) – creates Razorpay order via SDK, returns orderId+keyId. VerifyPaymentSignature(RazorpayVerifyRequest) – HMAC-SHA256 verification.

2.5.3 CheckoutSagaCommandsConsumer

Listens on the MassTransit-configured endpoint for ProcessPaymentCommand. On receipt: creates a Payment record (PaymentMode='Standard'), saves to DB, publishes PaymentProcessedEvent with Status='Success'. Also handles RefundPaymentCommand: records refund, publishes PaymentRefundedEvent.

2.6 ReturnsService

2.6.1 Models

Class	Key Properties	Notes
Return	Id, OriginalBillId, ProductId, Quantity, RefundAmount, Reason, Status ('Initiated'|'Approved'|'Refunded'|'Rejected'), ManagerApprovalNote, CustomerMobile, StoreId, Date	Status lifecycle managed by ReturnStateMachine

2.6.2 Return Saga (ReturnStateMachine)

State	Trigger Event	Action(s)	Next State
(Initial)	ReturnInitiatedEvent	Populate saga. Transition to AwaitingApproval.	AwaitingApproval
AwaitingApproval	ReturnApprovedEvent	Publish RefundPaymentCommand.	Refunding
AwaitingApproval	ReturnRejectedEvent	Publish FinalizeReturnCommand (Rejected).	Rejected (Finalized)
Refunding	PaymentRefundedEvent	Publish RestockItemCommand.	Restocking
Restocking	StockRestockedEvent	Publish FinalizeReturnCommand (Refunded) + OrderReturnedEvent.	Completed (Finalized)

2.7 NotificationService

2.7.1 Models and Hub

Class/Hub	Key Properties / Methods	Notes
Notification	Id, StoreId, Type, Title, Message, CreatedAt, IsRead	Persisted to NotificationDb
NotificationHub (SignalR)	JoinStoreGroup(storeId), SendNotification(user, message), OnConnectedAsync() – auto-joins Store_{storeId} group from JWT	Clients subscribe per store group
INotificationService	CreateAsync(notification), GetByStoreAsync(storeId)	Service layer for persistence

2.7.2 Event Consumers

Consumer	Subscribed Event	Notification Sent
UserRegisteredConsumer	UserRegisteredEvent	New staff registration notification to all store groups
StockAdjustedConsumer	StockAdjustedEvent	Stock adjustment alert to specific store group
OrderPlacedConsumer	OrderPlacedEvent	New order confirmation to store group
ReturnInitiatedConsumer	ReturnInitiatedEvent	Return request alert to store group (manager attention)
OrderReturnedConsumer	OrderReturnedEvent	Return completed notification to store group

3. Generic Repository Pattern

All services implement the following generic repository abstraction:

3.1 IGenericRepository<T>

Method Signature	Description
Task<T?> GetByIdAsync(int id)	Retrieve entity by primary key
Task<IEnumerable<T>> GetAllAsync()	Retrieve all entities (subject to global query filter)
Task<IEnumerable<T>> FindAsync(Expression<Func<T, bool>> predicate)	Filter entities by predicate
Task<T?> SingleOrDefaultAsync(Expression<Func<T, bool>> predicate)	Find single entity or null
Task<bool> AnyAsync(Expression<Func<T, bool>> predicate)	Check existence
Task AddAsync(T entity)	Add entity to context
void Update(T entity)	Mark entity as modified
void Delete(T entity)	Mark entity for deletion
Task SaveChangesAsync()	Persist changes to database

The GenericRepository<T> class uses a constructor-injected DbContext and delegates all operations to EF Core. Service-specific repositories inherit from GenericRepository<T> and add domain queries (e.g., GetWithRolesByEmailAsync in UserRepository).

4. Database Schema

4.1 IdentityDb

Table	Columns (PK*, FK^)	Constraints / Notes
Users	Id* (int), Email (nvarchar 100), PasswordHash (nvarchar 100), EmployeeCode (nvarchar 50), Otp (nvarchar), OtpExpiry (datetime2), IsEmailVerified (bit), VerificationOtp (nvarchar), VerificationOtpExpiry (datetime2), RefreshToken (nvarchar), RefreshTokenExpiry (datetime2)	Email: unique; BCrypt hash; RefreshToken stored server-side
Roles	Id* (int), Name (nvarchar 50)	Seeded: Admin, Manager, Cashier
Permissions	Id* (int), Code (nvarchar 100), Description (nvarchar 250)	Seeded with all permission codes from Permissions.cs
RolePermissions	Id* (int), RoleId^ (int), PermissionId^ (int)	Many-to-many bridge; seeded per role
UserStoreRoles	Id* (int), UserId^ (int), StoreId^ (nullable int), RoleId^ (int)	Null StoreId = Global Admin role
Stores (Identity)	Id* (int), StoreCode (nvarchar 50), Name (nvarchar 200)	Synced from AdminService via StoreCreatedEvent consumer

4.2 CatalogDb

Table	Columns (PK*, FK^)	Constraints / Notes
Categories	Id* (int, manual), Name (nvarchar 100), Description (nvarchar 500), IsActive (bit), StoreId (int)	Id not auto-generated; synced from AdminService
Products	Id* (int), Sku (nvarchar 50), Barcode (nvarchar 50), Name (nvarchar 200), MRP (decimal 18,2), SellingPrice (decimal 18,2), TaxCode (nvarchar 20), ReorderLevel (int, default 10), IsActive (bit), StockQuantity (int), CategoryId^ (int), StoreId (int)	Global query filter on StoreId; StockQuantity updated by saga consumers

4.3 OrdersDb

Table	Columns (PK*, FK^)	Constraints / Notes
Customers	Id* (int), Mobile (nvarchar 20, regex 10-digit), Name (nvarchar 100), CreatedAt (datetime2), StoreId (int)	Mobile validated on model level
Bills	Id* (int), BillNumber (nvarchar 50), Date (datetime2), StoreId (int), CashierId (int), CustomerMobile (nvarchar 20), TotalAmount (decimal 18,2), TaxAmount (decimal 18,2), Status (nvarchar 20), CustomerName (nvarchar max)	Status: Draft/Held/Finalized
BillItems	Id* (int), BillId^ (int), ProductId (int), ProductName (nvarchar 200), UnitPrice (decimal 18,2), Quantity (int), SubTotal (decimal 18,2), StoreId (int)	Cascade delete with Bill
CheckoutSagaStates	CorrelationId* (Guid), CurrentState (nvarchar), OrderId, StoreId, TotalAmount, CustomerMobile, PaymentId, PaymentStatus, CreatedAt, UpdatedAt, Items (JSON)	MassTransit saga persistence table; OrdersSagaDbContext

4.4 AdminDb

Table	Columns (PK*, FK^)	Constraints / Notes
Stores	Id* (int), StoreCode (nvarchar 50), Name (nvarchar 200), Location (nvarchar max), IsActive (bit)	Global query filter on IsActive; soft-delete
StaffMembers	UserId* (int, manual PK), Email (nvarchar), FullName (nvarchar), IsAssigned (bit), AssignedStoreId (int?), AssignedRole (nvarchar), RegisteredDate (datetime2)	UserId matches IdentityService User.Id
InventoryAdjustments	Id* (int), ProductId (int), Quantity (int), Reason (nvarchar), StoreId (int), Date (datetime2)	Positive = stock in; negative = stock out
SyncedOrders	OrderId* (int), StoreId, TotalAmount, TaxAmount, Date, CustomerMobile, Status	Populated from OrderPlacedEvent; used for dashboard aggregation
SyncedReturns	ReturnId* (int), OrderId, StoreId, RefundAmount, Date, CustomerMobile	Populated from OrderReturnedEvent
OnboardingSagaStates	CorrelationId* (Guid), CurrentState, UserId, Email, FullName, RoleName, StoreId, CreatedAt	MassTransit saga persistence; AdminSagaDbContext

4.5 PaymentDb

Table	Columns (PK*, FK^)	Notes
Payments	Id* (int), BillId (int), PaymentMode (nvarchar 20), Amount (decimal 18,2), ReferenceNumber (nvarchar 100), StoreId (int)	ReferenceNumber: Razorpay payment ID for digital payments

4.6 ReturnsDb

Table	Columns (PK*, FK^)	Notes
Returns	Id* (int), OriginalBillId (int), ProductId (int), Quantity (int), RefundAmount (decimal), Reason (nvarchar 500), Status (nvarchar 50), ManagerApprovalNote (nvarchar 500), CustomerMobile (nvarchar 20), StoreId (int), Date (datetime2)	Status lifecycle: Initiated/Approved/Refunded/Rejected
ReturnSagaStates	CorrelationId* (Guid), CurrentState, ReturnId, OrderId, StoreId, RefundAmount, ProductId, Quantity, CustomerMobile, CreatedAt, UpdatedAt	MassTransit saga persistence; ReturnsSagaDbContext

4.7 NotificationDb

Table	Columns (PK*, FK^)	Notes
Notifications	Id* (int), StoreId (int), Type (nvarchar), Title (nvarchar), Message (nvarchar), CreatedAt (datetime2), IsRead (bit)	Persisted via NotificationRepository; served by NotificationsController

5. API Endpoint Reference

5.1 IdentityService (Port 5001)

Method	Endpoint	Auth	Description
POST	/api/auth/login	Public	Submit email + password; triggers OTP send
POST	/api/auth/verify-login-otp	Public	Submit OTP; returns JWT + sets refresh cookie
POST	/api/auth/register	Public	Create new user account; sends email verification OTP
POST	/api/auth/verify-email	Public	Verify registration OTP
POST	/api/auth/resend-verification	Public	Resend email verification OTP
POST	/api/auth/resend-login-otp	Public	Resend login OTP
POST	/api/auth/send-otp	Public	Send password reset OTP
POST	/api/auth/reset-password	Public	Reset password using OTP
POST	/api/auth/google-login	Public	Google OAuth login with ID token
POST	/api/auth/refresh	Public (refresh cookie)	Refresh JWT access token using refresh cookie
POST	/api/auth/logout	Bearer (auth:logout)	Revoke refresh token; clear cookie
POST	/api/auth/sync-users	Bearer	Re-publish UserRegisteredEvent for all users
GET	/api/auth/users	Bearer	List all registered users
GET	/api/auth/stores	Bearer	List all stores (from IdentityDb)
GET	/api/stores/active	Public	List active stores
GET	/api/stores/{id}	Public	Get store by ID

5.2 CatalogService (Port 5002)

Method	Endpoint	Auth / Permission	Description
GET	/api/products	Bearer (catalog:view)	Get all products for authenticated store
GET	/api/products/{id}	Bearer (catalog:view)	Get product by ID
POST	/api/products	Bearer (catalog:manage)	Create new product
PUT	/api/products/{id}	Bearer (catalog:manage)	Update product
DELETE	/api/products/{id}	Bearer (catalog:delete)	Delete product
GET	/api/categories	Bearer (catalog:categories:view)	Get all categories for store
GET	/api/categories/{id}	Bearer (catalog:categories:view)	Get category by ID
POST	/api/categories	Bearer (catalog:categories:edit)	Create category
PUT	/api/categories/{id}	Bearer (catalog:categories:edit)	Update category
DELETE	/api/categories/{id}	Bearer (catalog:categories:edit)	Delete category

5.3 OrdersService (Port 5003)

Method	Endpoint	Auth / Permission	Description
GET	/api/bills	Bearer (orders:view_all)	Get all bills for store
GET	/api/bills/{id}	Bearer (orders:view)	Get bill by ID with items
POST	/api/bills	Bearer (orders:create)	Create or update draft bill
POST	/api/bills/cart/items	Bearer (orders:create)	Upsert cart items for a bill
POST	/api/bills/{id}/finalize	Bearer (orders:finalize)	Finalize bill; triggers CheckoutSaga
POST	/api/bills/{id}/hold	Bearer (orders:hold)	Hold bill

Method	Endpoint	Auth / Permission	Description
POST	/api/bills/{id}/void	Bearer (orders:void)	Void bill (manager action)
GET	/api/customers	Bearer	Get customers for store
GET	/api/customers/{id}	Bearer	Get customer by ID
POST	/api/customers	Bearer	Create new customer
PUT	/api/customers/{id}	Bearer	Update customer

5.4 AdminService (Port 5004)

Method	Endpoint	Auth / Permission	Description
GET	/api/dashboard/stats	Bearer (admin:reports:view)	Dashboard aggregates with optional storeId filter
GET	/api/stores	Bearer (admin:stores:view)	List active stores
GET	/api/stores/all	Bearer (all:all)	List all stores including inactive
POST	/api/stores	Bearer (admin:stores:manage)	Create store; publishes StoreCreatedEvent
PUT	/api/stores/{id}	Bearer (admin:stores:manage)	Update store; publishes StoreUpdatedEvent
DELETE	/api/stores/{id}	Bearer (admin:stores:manage)	Soft-delete store; publishes StoreUpdatedEvent
POST	/api/stores/{id}/restore	Bearer (admin:stores:manage)	Restore soft-deleted store
GET	/api/staff	Bearer (admin:stores:manage)	List all staff members
GET	/api/staff/pending	Bearer (admin:stores:manage)	List unassigned staff
POST	/api/staff/assign	Bearer (admin:stores:manage)	Assign staff to store + role; publishes StaffAssignedEvent
GET	/api/inventory	Bearer (admin:inventory:adjust)	Get inventory summary
GET	/api/inventory/adjustments	Bearer (admin:inventory:adjust)	Get adjustment history

Method	Endpoint	Auth / Permission	Description
POST	/api/inventory/adjust	Bearer (admin:inventory:adjust)	Record manual adjustment; publishes StockAdjustedEvent
GET	/api/reports/sales	Bearer (admin:reports:view)	Sales report (from/to query params)
GET	/api/reports/tax	Bearer (admin:reports:view)	Tax report (from/to query params)
GET	/api/categories	Bearer (catalog:categories:edit)	Manage categories via AdminService

5.5 PaymentService (Port 5005)

Method	Endpoint	Auth / Permission	Description
POST	/api/payments/create-order	Bearer (payments:create_order)	Create Razorpay order; returns orderId + keyId
POST	/api/payments/collect	Bearer	Record cash/internal payment; publishes PaymentProcessedEvent
POST	/api/payments/verify	Bearer (payments:verify)	Verify Razorpay payment signature
GET	/api/payments	Bearer	List payments for store
GET	/api/payments/{id}	Bearer	Get payment by ID

5.6 ReturnsService (Port 5006)

Method	Endpoint	Auth / Permission	Description
GET	/api/returns	Bearer (returns:view)	Get all returns for store
GET	/api/returns/{id}	Bearer (returns:view)	Get return by ID
POST	/api/returns	Bearer (returns:initiate)	Initiate a return; publishes ReturnInitiatedEvent; starts saga
POST	/api/returns/{id}/approve	Bearer (returns:approve)	Manager approves return; publishes ReturnApprovedEvent
POST	/api/returns/{id}/reject	Bearer (returns:approve)	Manager rejects return; publishes ReturnRejectedEvent

5.7 NotificationService (Port 5176)

Method	Endpoint	Auth	Description
GET	/api/notifications	Bearer (notifications:view)	Get all notifications for authenticated store
GET	/api/notifications/{id}	Bearer	Get notification by ID
POST	/api/notifications/{id}/read	Bearer	Mark notification as read
WS/GET	/notificationHub	Bearer (query param access_token)	SignalR WebSocket endpoint for real-time notifications

6. Key Request / Response Schemas

6.1 Authentication

DTO	Fields
LoginDto (Request)	{ Email: string, Password: string }
Login Response	{ requiresOtp: bool, message: string }
VerifyLoginOtpDto (Request)	{ Email: string, Otp: string }
AuthResponseDto (Response)	{ Token: string, Role: string, Email: string, StoreId: int, StoreCode: string?, Permissions: string[] }
RegisterDto (Request)	{ Email: string, Password: string }
GoogleLoginDto (Request)	{ IdToken: string, StoreId: int?, Role: string? }
TokenRequestDto (Request)	{ AccessToken: string } (RefreshToken from cookie)
ResetPasswordDto (Request)	{ Email: string, Otp: string, NewPassword: string }

6.2 Bills / Orders

DTO	Fields
BillDto (Request)	{ Id: int?, StoreId: int, CashierId: int, CustomerMobile: string, CustomerName: string, TotalAmount: decimal, TaxAmount: decimal, Items: BillItemDto[] }
BillItemDto	{ ProductId: int, ProductName: string, UnitPrice: decimal, Quantity: int, SubTotal: decimal }
Bill (Response)	{ Id, BillNumber, Date, StoreId, CashierId, CustomerMobile, TotalAmount, TaxAmount, Status, CustomerName, Items: BillItem[] }

6.3 Payment (Razorpay Flow)

DTO	Fields
RazorpayOrderRequest	{ Amount: decimal, Currency: string, Receipt: string }
RazorpayOrderResponse	{ OrderId: string, Amount: decimal, Currency: string, KeyId: string }
RazorpayVerifyRequest	{ RazorpayOrderId: string, RazorpayPaymentId: string, RazorpaySignature: string }

6.4 Returns

DTO	Fields
ReturnInitiationDto (Request)	{ OriginalBillId: int, ProductId: int, Quantity: int, Reason: string, CustomerMobile: string? }
ReturnActionDto (Request)	{ Note: string }
Return (Response)	{ Id, OriginalBillId, ProductId, Quantity, RefundAmount, Reason, Status, ManagerApprovalNote, CustomerMobile, StoreId, Date }

6.5 Dashboard

Field	Type	Description
TotalSales / TodaySales	decimal	Net sales (gross minus refunds)
GrossSales / RefundedAmount	decimal	Breakdown of total and refunds
TotalBills / TodayBills	int	Bill counts
SalesChangePercent	double	Percentage change today vs yesterday
LowStockAlerts / LowStockItems	int / LowStockItemDto[]	Products with stock < 10
RecentBills	RecentBillDto[]	5 most recent bills
HourlyTrend	HourlyTrendDto[]	Today's hourly sales breakdown
CategoryBreakdown	CategoryBreakdownDto[]	Store-wise sales percentage share

7. Detailed Workflow Sequences

7.1 Two-Factor Login Sequence

Step	Actor	Action / Message
1	Angular UI -> API Gateway	POST /gateway/auth/login {email, password}
2	API Gateway -> IdentityService	Forward to POST /api/auth/login (no auth required)
3	IdentityService (AuthService)	Load user by email; verify BCrypt password; generate 6-digit OTP; save OTP + 10-min expiry to User; send email via SMTP
4	IdentityService -> Angular UI	200 OK {requiresOtp: true, message: 'OTP_SENT'}
5	Angular UI -> API Gateway	POST /gateway/auth/verify-login-otp {email, otp}
6	IdentityService (AuthService)	Validate OTP and expiry; load UserStoreRoles with permissions; generate JWT (1hr) with permission claims; generate refresh token (64-byte); save refresh token to DB; set HttpOnly cookie
7	IdentityService -> Angular UI	200 OK {token, role, email, storeId, permissions} + Set-Cookie: refreshToken
8	Angular UI	Store JWT in memory; include as Authorization: Bearer {token} on all subsequent requests

7.2 Checkout Saga Sequence

Step	Actor	Action / Message
1	Cashier (Angular)	POST /gateway/orders/bills/cart/items (add products to cart)
2	OrdersService	Upsert Draft Bill + BillItems in OrdersDb
3	Cashier	POST /gateway/orders/bills/{id}/finalize
4	OrdersService (BillService)	Set Bill status = 'AwaitingCheckout'; Publish CheckoutInitiatedEvent to RabbitMQ
5	CheckoutStateMachine	State: Initial -> PaymentProcessing; Publish ProcessPaymentCommand to RabbitMQ
6	PaymentService (Consumer)	Create Payment record; Publish PaymentProcessedEvent {Status: 'Success'}
7	CheckoutStateMachine	State: PaymentProcessing -> StockDeductionProcessing; Publish DeductStockCommand

Step	Actor	Action / Message
8	CatalogService (SagaDeductStockConsumer)	Decrement StockQuantity for each item; Publish StockDeductedEvent
9	CheckoutStateMachine	State: StockDeductionProcessing -> Completed; Publish FinalizeOrderCommand + OrderPlacedEvent
10	OrdersService (SagaOrderCommandsConsumer)	Receive FinalizeOrderCommand; set Bill.Status = 'Finalized'
11	AdminService + NotificationService	Receive OrderPlacedEvent; sync to SyncedOrders; send real-time SignalR notification
12	Cashier (Angular)	200 OK confirmation; receipt generated

7.3 Return Workflow Sequence

Step	Actor	Action / Message
1	Cashier	POST /gateway/returns {originalBillId, productId, quantity, reason}
2	ReturnsService	Create Return record (Status: Initiated); Publish ReturnInitiatedEvent; ReturnSaga starts
3	ReturnStateMachine	State: Initial -> AwaitingApproval
4	NotificationService	Receive ReturnInitiatedEvent; notify manager's store group via SignalR
5	Manager	POST /gateway/returns/{id}/approve {note}
6	ReturnsService	Publish ReturnApprovedEvent
7	ReturnStateMachine	State: AwaitingApproval -> Refunding; Publish RefundPaymentCommand
8	PaymentService (Consumer)	Process refund; Publish PaymentRefundedEvent
9	ReturnStateMachine	State: Refunding -> Restocking; Publish RestockItemCommand
10	CatalogService (OrderReturnedConsumer)	Increment StockQuantity; Publish StockRestockedEvent
11	ReturnStateMachine	State: Restocking -> Completed; Publish FinalizeReturnCommand + OrderReturnedEvent
12	ReturnsService (Consumer)	Receive FinalizeReturnCommand; set Return.Status = 'Refunded'

Step	Actor	Action / Message
13	AdminService + NotificationService	Receive OrderReturnedEvent; update SyncedReturns; notify store group

7.4 Staff Onboarding Sequence

Step	Actor	Action / Message
1	Staff (New User)	POST /gateway/auth/register {email, password}
2	IdentityService	Create User (IsEmailVerified=false); send verification OTP; Publish UserRegisteredEvent
3	AdminService (UserRegisteredConsumer)	Create StaffMember record (IsAssigned=false) in AdminDb
4	AdminService (OnboardingStateMachine)	State: Initial -> ProfileCreationPending; Publish CreateStaffProfileCommand
5	Staff	POST /gateway/auth/verify-email {email, otp}; IsEmailVerified = true
6	Admin User	GET /gateway/admin/staff/pending (views unassigned staff)
7	Admin User	POST /gateway/admin/staff/assign {userId, storeId, role}
8	AdminService (StaffController)	Update StaffMember.IsAssigned = true; Publish StaffAssignedEvent
9	IdentityService (StaffAssignedConsumer)	Create UserStoreRole for user with assigned store and role
10	Staff	Can now log in and receive JWT with assigned role and permissions

8. Middleware and Cross-Cutting Implementation

8.1 ExceptionMiddleware

All services register app.UseExceptionMiddleware() which intercepts unhandled exceptions in the ASP.NET Core pipeline and returns standardized JSON error responses:

Exception Type	HTTP Status Code	Response Body
NotFoundException	404 Not Found	{ error: 'Not Found', message: '<exception message>' }
ValidationException / BusinessRuleException	400 Bad Request	{ error: 'Bad Request', message: '<exception message>' }
ConflictException	409 Conflict	{ error: 'Conflict', message: '<exception message>' }
Unhandled Exception	500 Internal Server Error	{ error: 'Internal Server Error', message: 'An unexpected error occurred' }

8.2 Authorization Middleware

The RetailPOS.Common.Authorization.ServiceExtensions.AddRetailPOSAuthorization() extension registers an ASP.NET Core AuthorizationPolicy for every permission constant defined in Permissions.cs. Each policy name equals the permission code string (e.g., 'orders:create'). The PermissionHandler reads the 'permission' claims from the authenticated JWT and checks for a match against the required policy permission code.

8.3 TenantProvider

Services requiring store-level data isolation inject ITenantProvider via Scoped DI. The TenantProvider.GetStoreId() method extracts the 'StoreId' claim from the current IHttpContext JWT principal. The DbContext's OnModelCreating applies a global query filter (e => e.StoreId == tenantProvider.GetStoreId()) on all tenant-aware entity sets. Admin endpoints bypass this filter using EF Core's IgnoreQueryFilters().

9. Shared Library Design (RetailPOS.Common and RetailPOS.Contracts)

9.1 RetailPOS.Common

Namespace / Class	Purpose
RetailPOS.Common.Authorization.Permissions	Static class containing all permission code string constants, organized by domain (System, Auth, Orders, Returns, Payments, Admin, Catalog, Notifications)
RetailPOS.Common.Authorization.PermissionRequirement	IAuthorizationRequirement implementation carrying the required permission code string
RetailPOS.Common.Authorization.PermissionHandler	IAuthorizationHandler<PermissionRequirement> that checks JWT 'permission' claims against the requirement
RetailPOS.Common.Authorization.ServiceExtensions	AddRetailPOSAuthorization(this IServiceCollection) – registers PermissionHandler and a named policy per permission code
RetailPOS.Common.Logging.SerilogConfiguration	ConfigureSerilog(this WebApplicationBuilder, serviceName) – configures Serilog with console + rolling file sinks, enriched with service name

9.2 RetailPOS.Contracts

All MassTransit message types are defined as C# interfaces in RetailPOS.Contracts, enabling type-safe publish/subscribe without requiring concrete implementations. MassTransit uses duck-typing to match anonymous objects to interface contracts at publish time.

Contract File	Interfaces Defined
OrderEvents.cs	CheckoutInitiatedEvent, OrderItemContract, OrderPlacedEvent, OrderItemEvent, OrderReturnedEvent, ReturnedItemEvent, ReturnInitiatedEvent, FinalizeOrderCommand, ReturnApprovedEvent, ReturnRejectedEvent, FinalizeReturnCommand
PaymentEvents.cs	PaymentProcessedEvent, PaymentFailedEvent, PaymentRefundedEvent
PaymentCommands.cs	ProcessPaymentCommand, RefundPaymentCommand
StoreEvents.cs	StoreCreatedEvent, StoreUpdatedEvent, StoreDeletedEvent
UserEvents.cs	UserRegisteredEvent, StaffProfileCreatedEvent, StaffAssignedEvent
UserCommands.cs	CreateStaffProfileCommand
InventoryEvents.cs	StockAdjustedEvent, StockDeductedEvent, StockDeductionFailedEvent, StockRestockedEvent, RestockItemCommand
CatalogCommands.cs	DeductStockCommand

Contract File	Interfaces Defined
CategoryEvents.cs	CategoryCreatedEvent, CategoryUpdatedEvent, CategoryDeletedEvent

10. Frontend Application (Angular 21 – pos-ui)

10.1 Technology Stack

Technology	Version	Purpose
Angular	21.x	SPA framework, routing, reactive forms
TypeScript	5.9.x	Typed JavaScript superset
RxJS	7.8.x	Reactive programming, HTTP observables
@microsoft/signalr	10.0.x	SignalR WebSocket client for real-time notifications
Angular CLI	21.x	Build, serve, test toolchain
npm	11.x	Package manager

10.2 API Communication

The Angular app communicates exclusively through the API Gateway (http://localhost:5000) using the /gateway/* path prefix. The JWT access token is stored in memory (not localStorage) and attached as an Authorization: Bearer header on every request. The refresh token is stored in an HttpOnly cookie managed by the browser and sent automatically to the /gateway/auth/refresh endpoint when the access token expires.

10.3 Real-Time Notifications

The Angular frontend establishes a SignalR WebSocket connection to /gateway/notificationHub, passing the JWT as access_token query parameter. On connection, the client is automatically added to Store_{storeId} group by the NotificationHub.OnConnectedAsync handler. Push events (new orders, low stock alerts, return requests) are received in real-time without polling.

10.4 Key Application Areas

- Authentication Module: Login, registration, OTP verification, password reset, Google OAuth
- POS Billing Module: Product search/scan, cart management, bill hold/finalize, receipt generation
- Customer Module: Customer lookup by mobile, create/update customer
- Catalog Module: Product and category management (Admin/Manager)
- Admin Dashboard: Real-time sales metrics, hourly trends, low stock alerts
- Staff Management: View pending staff, assign store + role
- Store Management: CRUD stores, restore deactivated stores
- Inventory Management: View stock levels, record adjustments
- Reports: Sales and tax reports with date range filtering
- Returns Module: Initiate returns (Cashier), approve/reject returns (Manager)
- Notifications Panel: Real-time notification feed from NotificationService

11. Logging Strategy

All microservices use Serilog configured via RetailPOS.Common.Logging.SerilogConfiguration. Log output targets:

- Console: Human-readable format [HH:mm:ss LVL] Message{NewLine}{Exception}
- Rolling File: Logs/{ServiceName}-{Date}.log, daily rolling interval

Log enrichment properties:

- Service: Service name (e.g., 'IdentityService', 'OrdersService')
- LogContext: Ambient context properties from Serilog.Context.LogContext.PushProperty
- ExceptionDetail: Full exception stack traces via Serilog.Exceptions

Log level overrides:

- Microsoft.* namespaces: Suppressed to Warning to reduce framework noise
- Microsoft.Hosting.Lifetime: Set to Information for startup/shutdown messages
- Application code: Verbose (all levels)

12. Implementation Assumptions and Notes

ID	Finding / Assumption	Impact
LLD-01	RefundAmount in ReturnSaga is calculated as quantity * 10 (placeholder). The actual refund should be derived from the original payment record for the bill.	Low – functional for demo; must be fixed for production
LLD-02	DashboardService uses IgnoreQueryFilters() and loads all synced orders/returns into memory before filtering. For large datasets this will have performance implications.	Medium – requires pagination or SQL-level aggregation
LLD-03	ReportService proxies directly to OrdersService internal URL (http://127.0.0.1:5003/api/bills). This bypasses the API Gateway and creates direct service coupling.	Medium – acceptable in monorepo dev; should use service discovery in production
LLD-04	OnboardingStateMachine references StaffProfileCreatedEvent but no explicit publisher of this event was found in the codebase. The SagaUserOnboardingConsumer in AdminService may handle the CreateStaffProfileCommand and implicitly emit the state.	Low – saga may stall if event is never published; needs verification
LLD-05	IdentityService and NotificationService enable full CORS (AllowAnyOrigin), while other services delegate CORS to the API Gateway. This inconsistency should be unified in production.	Low – no functional issue in dev
LLD-06	JWT audiences are configured as a comma-separated list via Jwt:Audiences configuration key. All services must share the same audience list to accept tokens issued by IdentityService.	Medium – configuration must be synchronized across all services
LLD-07	BCrypt.Verify is used for password validation with auto-migration of legacy plaintext passwords on first login. This is a one-time migration path for existing users.	Low – functional and secure
LLD-08	StockQuantity in CatalogDb is maintained by event consumers, not calculated from a ledger. If a consumer misses a message (e.g., due to RabbitMQ downtime), stock figures may become inconsistent.	High – recommend idempotent consumers with retry + dead-letter queues in production